Week 6: For Loops and Array Algorithms

ID1063 • Introduction to Programming in C

Instructor: Rakesh Venkat

Lectures 11 and 12

Recap from Week 5

- A nested loop completes all inner iterations for one outer iteration.
- An array stores related values at indices 0 through n - 1.
- A while loop can be used to read and process every element of an array.
- Array bounds should be carefully adhered to avoid unexpected results

Recap: Find the output

```
int a[4] = {3, 8, 2, 7};
int sum = 0;

for (int i = 0; i < 4; i = i + 2) {
    sum = sum + a[i];
}

printf("%d", sum);
```

Which indices are used? What is printed?

This week

- The for loop: a compact form of a counting loop.
- Using for loops to read and process arrays.
- Some array algorithms: statistics, counting and search.

For loops

Structure of Loops

Three essential parts of a loop:

1. Initial value
2. Termination
3. Update

- Use a for loop when the initial value, condition, and update belong together.
  - The loop variable is commonly an index such as i.

The for loop syntax

```
for (initialization; condition; update) {
    statements;
}
```

```
for (int i = 0; i < 5; i = i + 1) {
    printf("%d ", i);
}
```

Execution order: initialize once → test → body → update → test again.

The same loop: while and for

```
int i = 0;
while (i < n) {
    printf("%d ", values[i]);
    i = i + 1;
}
```

```
for (int i = 0; i < n; i = i + 1) {
   printf("%d ", values[i]);
  }
```

Both visit indices 0 through n - 1.

- Can you think of an example where a while loop could be preferred over a for loop?

When a while loop is preferred

```
int reading;
int sum = 0;

scanf("%d", &reading);
while (reading != -1) {
    sum = sum + reading;
    scanf("%d", &reading);
}
```

Use while when the number of repetitions is not known in advance.
Here, -1 is a sentinel value that stops the input.

Example 1: Reading an array

```
int values[50];
int n;

scanf("%d", &n);
for (int i = 0; i < n; i = i + 1) {
    scanf("%d", &values[i]);
}
```

Note the terminating criterion

Q: Can you print the value of i after the loop? What value does it take?

Boundary cases in a for loop

```
for (int i = 0; i <= n; i = i + 1) {
    scanf("%d", &values[i]);
}
```

What is the output here?

What is the output here?

```
for (int i = 2; i <= 8; i = i + 3) {
    printf("%d ", i);
}
```

i = 2 → ok → prints 2
i = 5 → ok → prints 5
i = 8 → ok → prints 8
i = 11 → not ok → exits loop

Trace i = 2, 5, 8, 11. The output is 2 5 8.

Find the output

```
int n = 5;
int a[5] = {2, 4, 6, 8, 10};
```

```
for (int i = 0; i < n; i = i + 2) {
    printf("%d ", a[i]);
}
```

→ prints even-indexed positions

```
for (int i = n - 1; i >= 0; i = i - 1) {
    printf("%d ", a[i]);
}
```

→ i = 4 → prints a[4]
i = 3 a[3]
i = 2 a[2]
i = 1
i = 0 a[0]

Find the output

```
int total = 0;
for (int i = 1; i <= 10; i = i + 1) {
    if (i % 2 == 0) {
        total = total + i * i;
    }
}
printf("%d", total);
```

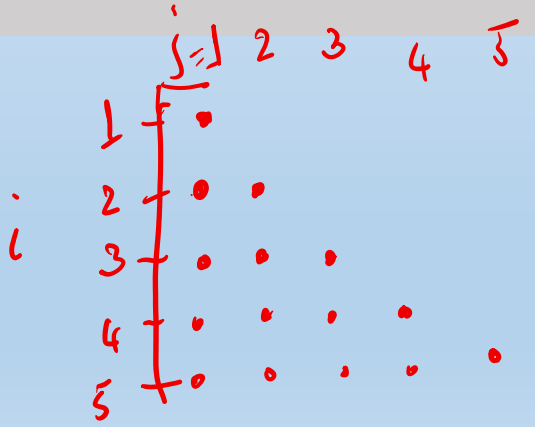
Ans: Sum of squares of even integers $2^2 + 4^2 + 6^2 + 8^2 + 10^2$

Nested for loops

int n=5;

```
for (int i = 1; i <= n; i = i + 1) {
    for (int j = 1; j <= i; j = j + 1) {
        printf("%d ", j);
    }
    printf("%c", 10);
}
```

For n = 4, predict the output.

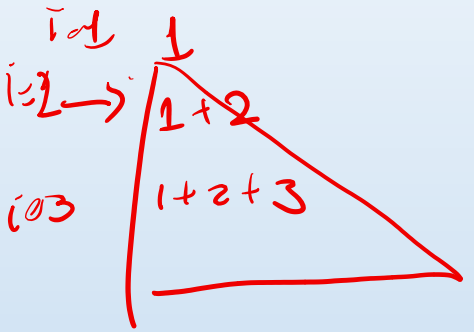


Find the output

```
int count = 0;


for (int i = 1; i <= 3; i = i + 1) {
    for (int j = 1; j <= i; j = j + 1) {
        count = count + j;
    }
}


printf("%d", count);
```



Trace the contribution of each outer-loop iteration.

Lec 12: Strings, more Arrays

Recap Maximum and its index

```
int maximum = marks[0];
int max_index = 0;

for (int i = 1; i < n; i = i + 1) {
    if (values[i] > maximum) {
        maximum = values[i];
        max_index = i;
    }
}
```

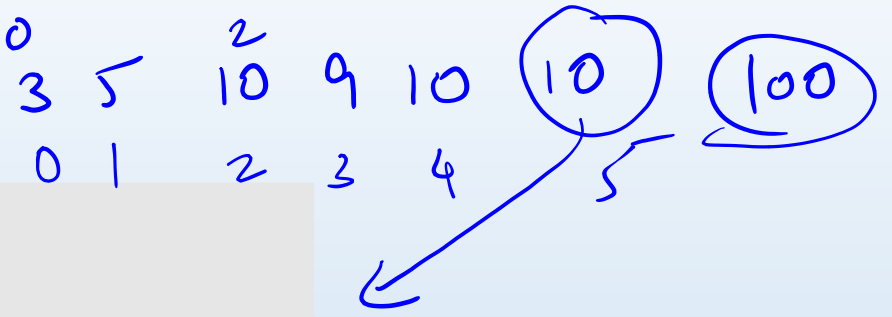
Goal: Given an array, find the maximum and its index

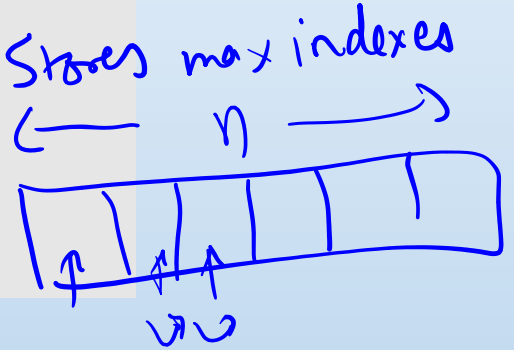
Q:

Ties

```
for (i = 0; i < 6; i++) {
if (values[i] >=maximum) {
    maximum = values[i];
    max_index = i;
}

}
```





Q1: For repeated max elements, what is the output?

Q2: Can you think of how to print *all* indices that attain the maximum?

① Find max value ② Find indices that hold max

Character arrays and strings

Character arrays

- A character array stores characters at indices 0, 1, 2, ...
- Example: char name[20]; reserves space for 20 characters.
- A string is a character array with a special terminating character.

A string ends with '\0'

```
char word[4];

word[0] = 'c';
word[1] = 'a';
word[2] = 't';
word[3] = '\0';
```

word = string

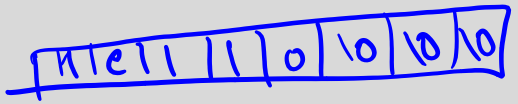
- The null character (\0) marks the end of the string
- It is **not the character '0', or a space**.
- The array word could have been larger than 4 in size. The string ends at the first '\0' character.

Initializing Strings: Three ways

```
char str[] = "Hello";
// Memory size: 6 bytes ('H', 'e', 'l', 'l', 'o', '\0'), Array size = 6.

char str[10] = "Hello";
// First 5 elements are 'H', 'e', 'l', 'l', 'o' // Remaining 5 elements
are automatically filled with '\0'

char str[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```




```
//A wrong version:
char str[10];
str = "Hello"; // ERROR: Array types are not assignable
```

We cannot say str = some string, assignment is not possible in this way with arrays.
Just like integer arrays cannot be assigned.

Getting a string Input

```
char word[20];

scanf("%19s", word);
printf("%s\n", word);
```

- Why %19s? : to make sure not more than 19 characters are read, since array size is 20.
- What happens if we give “Hello World” as input?
  - A: String stops at Hello, since input is taken only till the first space.

- IMPORTANT: Do not use **&** in front of the variable in scanf
  - Reason will be clear when we study pointers

String Input with spaces

```
char str[50];

printf("Enter a full sentence: ");
// %49[^\n] reads up to 49 characters until a newline is hit
scanf("%49[^\n]", str);

printf("You entered: %s\n", str);
```

Important points about String Input

- **Buffer Overflow Protection**
  - Always specify size of the buffer to take string input

- Ensure the character array is large enough for the null terminator ('\0').
  - For example, an array of size 50 can store at most 49 user-entered characters.

Finding Length of a String

```
int word[]=“abc”
int length = 0;

while (word[length] != '\0') {
    length = length + 1;
}
```

Stop at \0, not at the end of the array: a string may use fewer array locations.

Count vowels

- Count the number of vowels in a given input string

Count vowels

```
int vowels = 0;

for (int i = 0; word[i] != '\0'; i = i + 1) {
    if (word[i] == 'a' || word[i] == 'e' ||
        word[i] == 'i' || word[i] == 'o' ||
word[i] == 'u') {
        vowels = vowels + 1;
    }
}
```

Is a word a palindrome?

```
Input:  level
Output: YES

Input:  array
Output: NO
```

Read a lowercase word. Print YES only if it reads the same from both ends.

Outline: Split the steps into logical parts:

A) Task 1: Find the length of the string, store it in a variable, say, len
B) Task 2: Check Palindrome condition: str[i]==str[len-i] ? Take care of boundary conditions.

Longest run of one character

```
Input:  aaabbccccda
Output: 4

Longest run: cccc
```

Read a lowercase word and find the largest number of consecutive equal characters in one traversal.

Common string mistakes

- char word[20] stores at most 19 typed characters: reserve one for \0.
- %s reads one word—not a complete line with spaces.
-
- A string loop stops at word[i] != '\0'.
- Use %c for one character and %s for a string.

String.h

- (Not covered in Lec12, but will be covered in Lec13)
- Provides the string library and many useful string functions

Function	Purpose	Example / Description
strlen(str)	Returns the length of the string (excluding '\0').	size_t len = strlen("Hello"); // Returns 5
strcpy(dest, src)	Copies src string to dest (including '\0').	Potential buffer overflow if dest isn't large enough.
strncpy(dest, src, n)	Copies up to n characters from src to dest.	Safer alternative to strcpy.
strcat(dest, src)	Appends src to the end of dest.	dest must have enough memory to hold the combined string.
strncat(dest, src, n)	Appends up to n characters from src to dest.	Safer alternative to strcat.
strcmp(str1, str2)	Compares two strings lexicographically.	Returns 0 if equal, <0 if str1 < str2, >0 if str1 > str2.
strncmp(str1, str2, n)	Compares up to n characters of two strings.	Useful for partial matching or safe comparisons.

Example code

```
#include <stdio.h>
#include <string.h>

int main() {
    char dest[20] = "Hello";
    char src[] = " World!";

    strcat(dest, src);
    printf("%s\n", dest); // Output: Hello World!

    return 0;
}
```

Some good coding practices

Indent to show block structure

```
for (int i = 0; i < n; i = i + 1) {
    if (marks[i] >= 50) {
        sum = sum + marks[i];
        count = count + 1;
 }
}
```

After {, move one level right. Before }, return one level left. Use four spaces for each level.

One loop, one purpose

```
int matches = 0;

for (int i = 0; i < n; i = i + 1) {
    if (values[i] == target) {
        matches = matches + 1;
    }
}
```

Initialise before the loop, update inside the relevant condition, and keep i only as the index.

Test boundary cases

- Trace one small input before running the program.
- For an if/else, test inputs that make each branch run.
- For a loop, check 0 iterations, 1 iteration, and the last valid index.
- For array algorithms, test n = 1 and repeated values.
- Compile and fix the first error before changing anything else.